

different ways to create stream in java with example

Certainly! Here are several different ways to create a Stream in Java along with examples:

1. ****From a Collection:****

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);  
Stream<Integer> streamFromCollection = numbers.stream();
```

2. ****From an Array:****

```
String[] array = {"apple", "banana", "orange"};  
Stream<String> streamFromArray = Arrays.stream(array);
```

3. ****Using Stream.of() for Individual Elements:****

```
Stream<String> streamOfElements = Stream.of("apple", "banana", "orange");
```

4. ****Generating Streams with Stream.generate():****

```
Stream<String> generatedStream = Stream.generate(() -> "Hello");
```

5. ****Iterating with Stream.iterate():****

```
Stream<Integer> iteratedStream = Stream.iterate(0, n -> n + 2).limit(5);
```

6. ****From a Range of Values:****

```
IntStream rangeStream = IntStream.range(1, 6);
```

7. ****From a Range of Values (Inclusive):****

```
IntStream rangeClosedStream = IntStream.rangeClosed(1, 5);
```

8. ****Creating an Empty Stream:****

```
Stream<Object> emptyStream = Stream.empty();
```

How to iterate/print a stream:

To iterate or print the elements of a Stream in Java, you can use various methods provided by the Stream API. Here are a few common approaches:

1. ****Using forEach():****

You can use the `forEach()` method to perform an action for each element in the stream.

```
Stream<String> stream = Stream.of("apple", "banana", "orange");
stream.forEach(System.out::println);
```

2. ****Using forEachOrdered():****

This method is similar to `forEach()`, but it guarantees that the elements are processed in the order specified by the stream.

```
Stream<String> stream = Stream.of("apple", "banana", "orange");
stream.forEachOrdered(System.out::println);
```

3. ****Converting to List and Using Iterator:****

You can convert the stream to a List and then use an Iterator to iterate over the elements.

```
Stream<String> stream = Stream.of("apple", "banana", "orange");
List<String> list = stream.collect(Collectors.toList());
Iterator<String> iterator = list.iterator();
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
```